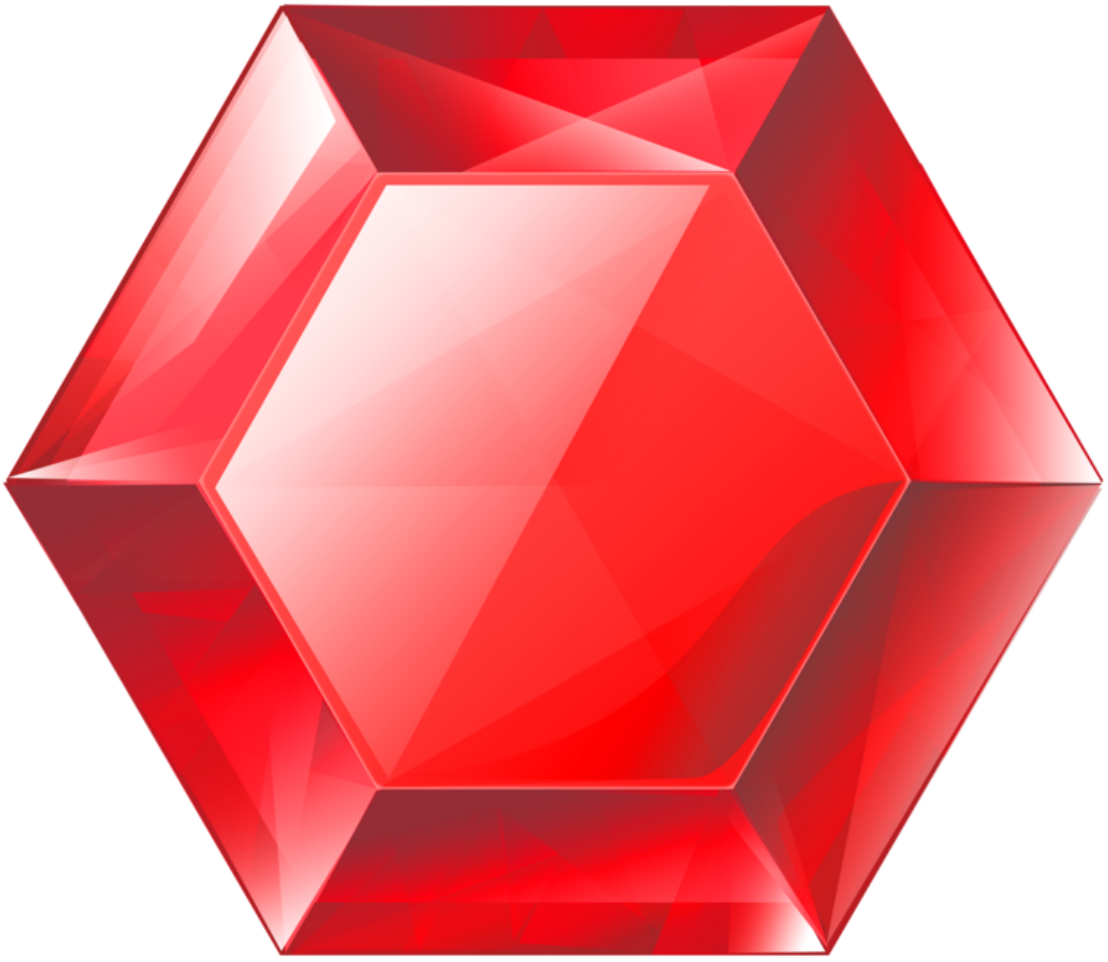
gemstone-rs Feature Map

Crates, examples, docs, and gemstone-py parity references



Generated from the Markdown sources in gemstone-rs/docs.

Contents

gemstone-rs Feature Map

This map is the Rust equivalent of `gemstone-examples plan3-map` in `gemstone-py`: it ties each feature stream to the crates, examples, docs, and Python reference point that inspired it.

Run it from the CLI:

```
gemstone-rs hello
gemstone-rs compare gemstone-py
gemstone-rs compare gemstone-py --gaps
gemstone-rs examples map
gemstone-rs examples map --json
gemstone-rs examples scaffold quickstart ./gemstone-rs-quickstart
gemstone-rs examples scaffold codegen_workflow ./gemstone-rs-codegen-workflow
gemstone-rs examples scaffold profile_codegen_workflow ./gemstone-rs-profile-codegen
gemstone-rs examples scaffold generated_wrapper_app ./gemstone-rs-generated-wrapper
gemstone-rs examples scaffold session_worker_pool ./gemstone-rs-worker-pool
gemstone-rs examples scaffold axum_service ./gemstone-rs-axum-service
gemstone-rs examples scaffold actix_service ./gemstone-rs-actix-service
```

`hello` is the no-live sanity check that mirrors `gemstone-examples hello`. `compare gemstone-py` prints a compact version of the Python/Rust comparison guide. `compare gemstone-py --gaps` prints the remaining parity gaps with next actions and verification commands. `examples scaffold` creates standalone Cargo projects from installed templates for quickstart, browser, BridgeRoot mapping, derive mapping, generated wrappers, live discovery, profile-driven codegen, standard HTTP, worker-pool, Axum, and Actix workflows. JSON output is intended for CI, docs checks, and editor tooling.

Streams

Stream	Feature	Rust surface	Examples	Docs	ge
1	Runtime and GCI loading	`gemstone-gci`, `gemstone-rs::Config`	`hello`, `hello_gemstone`, `quickstart`	`docs/setup-guide.md`, `docs/performance-safety.md`	`g` na
2	Safe sessions and transactions	`gemstone-rs::Session`, `SessionWorker`, `SessionWorkerPool`, `TransactionGuard`	`quickstart`, `transactions`, `session_worker`, `session_worker_pool`, `live_smoke_cookbook`	`docs/user-manual.md`, `docs/cookbook.md`	`G` `S` tra

Stream	Feature	Rust surface	Examples	Docs	ge
3	Browser and inspection	`gemstone-rs::browser`, CLI `browse`, `gemstone-rs-explorer`	`browser`, `tooling/cli-browser-walkthrough.md`	`docs/user-manual.md`, `docs/explorer.md`	g p da
4	OOP and value handling	`Oop`, `Value`, export-set helpers	`oop_values`	`docs/user-manual.md`, `docs/performance-safety.md`	M ty
5	BridgeRoot object mapping	`gemstone-rs::bridge`, `gemstone-rs-macros`	`bridge_root_mapping`, `derive_mapping`, `generated_mapping_app`	`docs/object-mapping.md`, `docs/cookbook.md`	S P ex
6	Typed codegen	`gemstone-rs::codegen`, CLI `codegen` and `profile`	`codegen_preview`, `codegen_workflow`, `codegen_discover`, `profile_codegen_workflow`, `generated_wrapper_app`	`docs/codegen.md`, `docs/profile-schema.md`	g t co
7	Explorer workflow	`gemstone-rs-explorer`	`tooling/explorer.md`	`docs/explorer.md`, `docs/screenshots.md`	p da
8	VS Code workbench	`vscode-gemstone-rs-workbench`	`tooling/vscode-workbench.md`	`docs/vscode-workbench.md`	g
9	Rust web services	`gemstone-rs::SessionWorkerPool` plus checked Axum/Actix services and installed scaffolds	`session_worker`, `session_worker_pool`, `http_service`, `examples/axum-service`, `examples/actix-service`, `examples scaffold`	`docs/examples-guide.md`, `docs/cookbook.md`	Fa ex

Stream	Feature	Rust surface	Examples	Docs	ge
			`session_worker_pool/` `axum_service/` `actix_service`		
10	Release and verification	`scripts`, `Makefile`, GitHub Actions	Release verification commands	`docs/release-checklist.md`	Py VS
11	Shared native core	`gemstone-gci`, `gemstone-rs`, future `gemstone-py-native` wrapper	Shared-core integration plan	`docs/shared-core-integration.md`	g

What This Says Compared With gemstone-py

`gemstone-rs` is strongest where Rust is naturally valuable: direct native GemStone access, explicit OOP/value handling, typed generated wrappers, BridgeRoot mapping, and CLI/explorer tooling that can run without Python in the process.

`gemstone-py` remains stronger where Python already has mature product shape: broader web framework adapters, async examples, the database explorer, package extras, and broader release surfaces.

The projects should converge by sharing the Rust native core underneath `gemstone-py-native`, while keeping the Python and Rust APIs idiomatic for their respective users.

> This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.